

EcoOrigem

RASTREABILIDADE DE PRODUTOS DA BIOECONOMIA AMAZÔNICA COM BLOCKCHAIN

Oficina de Desenvolvimento de Sistemas III

ANA BEATRIZ MACIEL NUNES
FERNANDO LUIZ DA SILVA FREIRE
JULIANA BALLIN LIMA

Contexto

Problema e objetivo do projeto

O problema

Na cadeia da bioeconomia, cada organização mantém seus próprios registros. Isso dificulta verificar a integridade do histórico e identificar quem realizou cada etapa.

O objetivo

O EcoOrigem registra o ciclo do lote em um contrato público. A MetaMask identifica os participantes, o Solidity controla perfis e estados, e a Sepolia confirma transações e eventos.

Produtor → Beneficiador → Transportador → Distribuidor → Consumidor

Primeira Entrega

Base didática validada em ambiente local

Blockchain local em Python

Blocos encadeados, prova de trabalho, nonce, SHA-256, raiz de Merkle e carteiras com assinatura digital.

Regras de negócio

Perfis, custódia e ordem obrigatória do produtor ao distribuidor.

Aplicação web

Nó HTTP, interface, Docker e mensagens amigáveis para a demonstração.

Qualidade

268 testes, cenários válidos e inválidos, relatório técnico e evidências.

Resultado: conceito e fluxo comprovados

Segunda Entrega

Melhorias e novas implementações da dApp pública



Contrato Solidity

Perfis, estados, eventos, erros e taxa pagável.



Ethereum Sepolia

Estado público e transações consultáveis.



MetaMask

Conexão, assinatura e confirmação de transações.



Frontend Web3.js

Leituras, escritas, eventos e erros tratados.



Hash e IPFS

Prova on-chain com arquivo mantido fora da rede.



Vercel e testes

HTTPS, Remix, testes locais e auditoria de segredos.

Do ambiente local para uma rede Ethereum pública de teste

IDEs, frameworks e serviços

Três etapas: o que se integra com o quê e em que ordem



1 Construir e testar

Solidity — regras, perfis, eventos

escreve o .sol →

Remix IDE — compila e gera a ABI

roda testes →

Remix VM — testa sem gastar ETH

automatiza →

Node.js — testes com Ganache

envia o código →

GitHub — versiona e abre no Remix

2 Implantar na Sepolia

PoW Faucet — SepoliaETH gratuito

envia ETH →

MetaMask — guarda chaves e saldo

conecta →

Remix IDE — Injected Provider

deploy assinado →

Sepolia — executa o contrato

bloco e hash →

Etherscan — verifica código e hash

3 Publicar e usar

Vercel — site estático HTTPS

carrega →

Web3.js 4.x — lê e envia pela ABI

pede assinatura →

MetaMask — assina cada ação

envia tx →

Sepolia — grava e emite eventos

hash + URI →

IPFS — arquivo fora da rede

Fluxo completo da dApp

Da ação no navegador à evidência pública



Máquina de estados do lote



Operações fora do perfil ou da ordem são recusadas sem alterar o estado.



Demonstração ao vivo

O que será mostrado e onde

01 **Conectar e autenticar a carteira**
Vercel + MetaMask

02 **Ler totais, perfil e lotes com call**
Frontend

03 **Cadastrar lote com taxa em SepoliaETH**
Frontend + MetaMask

04 **Acompanhar hash, bloco e confirmação**
Frontend + Etherscan

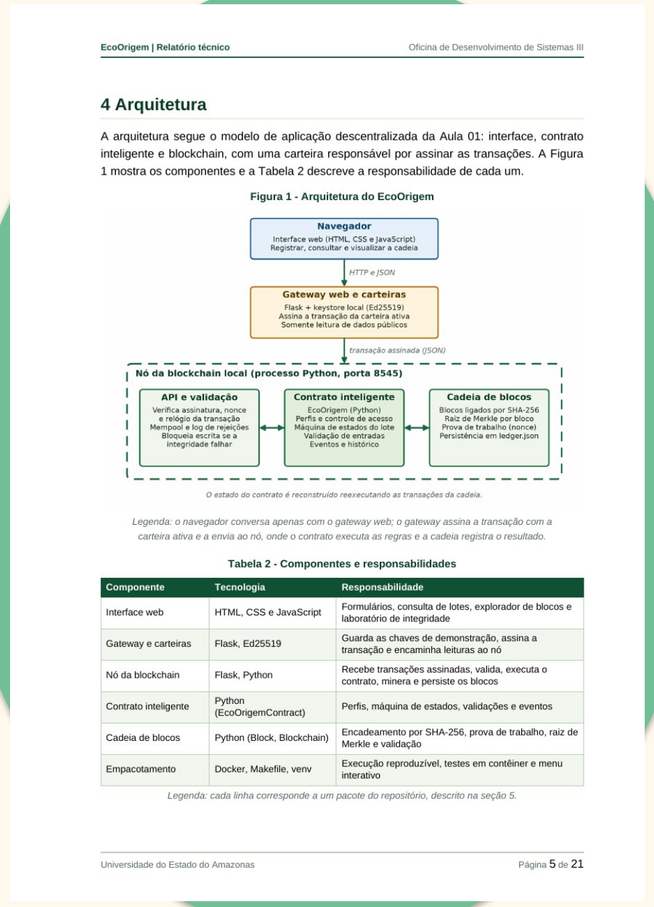
05 **Provocar erro de perfil e conferir evento**
Frontend + Etherscan

Plano B: vídeo e capturas reais, sem novo deploy durante a apresentação.



Bônus: relatório de desenvolvimento

Evidência técnica da evolução do projeto



4 Arquitetura

A arquitetura segue o modelo de aplicação descentralizada da Aula 01: interface, contrato inteligente e blockchain, com uma carteira responsável por assinar as transações. A Figura 1 mostra os componentes e a Tabela 2 descreve a responsabilidade de cada um.

Figura 1 - Arquitetura do EcoOrigem

Legenda: o navegador conversa apenas com o gateway web; o gateway assina a transação com a carteira ativa e a envia ao nó, onde o contrato executa as regras e a cadeia registra o resultado.

Tabela 2 - Componentes e responsabilidades

Componente	Tecnologia	Responsabilidade
Interface web	HTML, CSS e JavaScript	Formulários, consulta de lotes, explorador de blocos e laboratório de integridade
Gateway e carteiras	Flask, Ed25519	Guarda as chaves de demonstração, assina a transação e encaminha leituras ao nó
Nó da blockchain	Flask, Python	Recebe transações assinadas, valida, executa o contrato, minera e persiste os blocos
Contrato inteligente	Python (EcoOrigemContract)	Perfis, máquina de estados, validações e eventos
Cadeia de blocos	Python (Block, Blockchain)	Encadeamento por SHA-256, prova de trabalho, raiz de Merkle e validação
Empacotamento	Docker, Makefile, venv	Execução reproduzível, testes em contêiner e menu interativo

Legenda: cada linha corresponde a um pacote do repositório, descrito na seção 5.

Relatório técnico

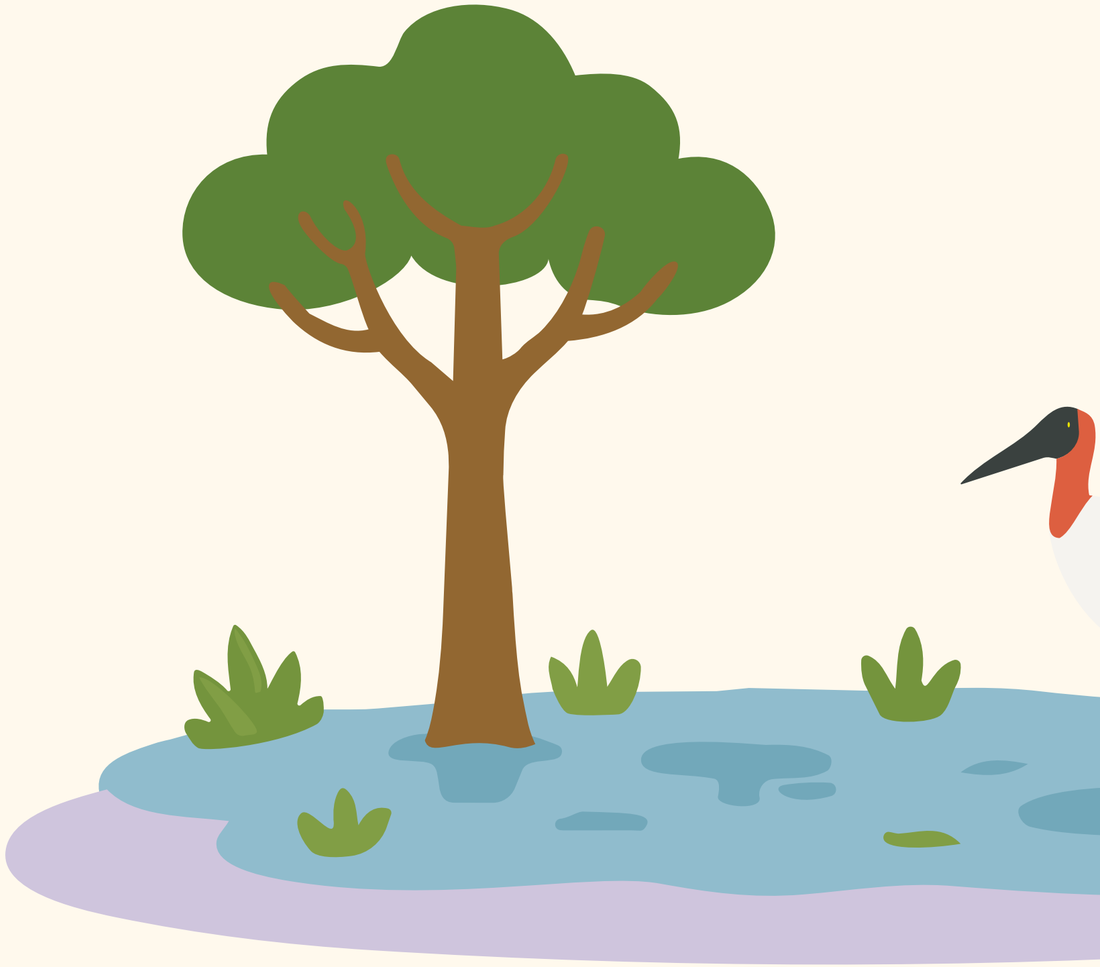
21 páginas preservadas em PDF e DOCX.

Conteúdo

Arquitetura, decisões, testes, segurança e limitações.

Valor para a Entrega 2

Documenta a base que originou a migração para Ethereum.



docs/relatorio/relatorio-desenvolvimento-ecoorigem.pdf

EcoOrigem

Obrigada

Perguntas?

github.com/JulianaBallin/EcoOrigem

UEA
UNIVERSIDADE
DO ESTADO DO
AMAZONAS